

Performance Testing Report

Computer Systems and Network Engineering – Final Year Research Project

Project Title: Cargo Mate - Smart Robotic Inventory System with Cloud Integration (Cloud Integration Subsystem)

Student Name: R.A.G Perera

Student ID: IT22903242

Supervisor: Mr. Uditha Dharmakeerthi

Date: 29/04/2026

1. Introduction and Objective

The fast advancements that have been experienced in cloud computing, real-time data processing, and intelligent analytics have greatly changed today's warehouse management systems. The problems that have been associated with conventional inventory management include poor real-time update capability, inefficiencies in scaling, and inadequate monitoring capacity. This research paper seeks to analyse the Cloud-Integrated Intelligent Warehouse Inventory Management System which is equipped with features like real-time database synchronizing, Role Based Access Control (RBAC), intelligent analytics, digital twin display, and cloud-based monitoring.

This performance test report will aim at analysing how efficiently, effectively, and reliably the developed system can perform under different scenarios. There are many features involved in the design of this system including; the Firebase Realtime Database used for synchronizing of data in real time, the web-based RBAC for enabling user interactions, and Grafana which is a real-time monitor among others.

Performance testing is an integral part of the system development process in terms of applications implemented in cloud environments, where there are interactions between several users. In the case of a warehouse, the poor system performance may have a direct effect on such processes as inventory accuracy, efficiency of processes, and decision making. Thus, it is necessary to verify that the system can operate under several users at a time, provide real-time updates, and be stable in both regular and stressful conditions.

The purpose of the current research paper is to investigate the performance features of the implemented system by using various types of tests aimed at evaluating the system behaviour under different conditions. It should be mentioned that the performance testing aims to identify whether the system can operate effectively under various loadings and determine possible bottlenecks.

In general, the results obtained during the testing process will provide significant information regarding the operation of the system, which can help improve its performance. Overall, it should be stated that the purpose of this paper is to ensure that the designed solution will be effective enough in the real world.

2. Performance Testing Environment Setup

Testing environment describes the configuration of hardware, software, and network for testing the system.

An appropriate testing environment makes the output obtained from testing credible and realistic.

In this case, the testing environment was configured to resemble a cloud-based warehouse system where there is simultaneous access to the dashboard, database, and monitoring features by different users

Hardware Configuration- Testing involved the use of both cloud servers and client machines:

Application deployment took place on cloud servers, facilitating real-time operations.

Tests were done on a powerful laptop (Intel i7, 16 GB RAM) to mimic multi-users.

Multiple browser instances were opened for simulating multi-users accessing the dashboard.

These methods of testing allowed the system to be put through tests just like in an actual warehouse with multi-users.

Software Configuration- The software tools that were used in testing include the following:

Front-end: Dashboard (React / browser interface)

Back-end: Firebase and cloud services for data processing

Database: Firebase Realtime Database for instant synchronization

Monitoring tool: Grafana for real-time performance monitoring

The software tools were chosen because they can be used in real-time applications, cloud computing environments, and performance analysis.

Network Conditions- The system was tested under different network conditions to evaluate performance:

- Normal Network: Stable internet connection with low latency
 - Moderate Load Network: Multiple users accessing simultaneously
 - Degraded Network: Simulated delays and slower response times
- Testing under different network conditions is important because cloud-based systems depend heavily on internet performance.

3. Tools and Frameworks Used

In performance testing, selecting the right tools and frameworks is essential to accurately simulate user behaviour, measure system performance, and analyse results. In this project, a combination of industry-standard tools was used to evaluate different aspects of the cloud-integrated warehouse system.

Firestore Realtime Database

Firestore was used as the cloud database system for real-time data synchronization.

- Stores inventory and system data
- Supports real-time updates using WebSocket
- Tested for latency and synchronization speed

Grafana

Grafana was used for real-time monitoring and visualization of system performance.

- Displays system metrics (CPU, memory, requests)
- Monitors system behaviour during testing
- Helps detect performance issues

Prometheus

Prometheus works with Grafana to collect performance data from the system.

- Collects metrics from server and application
- Stores data for analysis
- Sends data to Grafana dashboards

Python

Python was used for data analysis and visualization after testing.

- Processes test results
- Creates graphs and charts
- Helps interpret performance data

4. Testing Methodology and Execution

The testing methodology is the formalized testing approach used to measure the system's performance, whereas execution describes the manner in which the tests have been done. Good methodologies ensure that the results obtained are accurate and consistent.

For the current project, a sound performance testing process has been adopted to measure the system behaviour in a realistic environment.

4.1 Test Scenarios

Different test scenarios were designed to evaluate how the system performs under various conditions:

- Baseline Test – Small number of users (normal usage)
- Normal Load Test – Moderate number of users
- Peak Load Test – High number of concurrent users
- Stress Test – System pushed beyond its limits
- Endurance (Soak) Test – Long-duration testing
- Spike Test – Sudden increase and decrease in users
- Component Testing – Testing specific parts (dashboard, database, alerts)

4.2 Execution Procedure

The following steps were followed for each test:

Step 1: Environment Preparation

- System was configured and database was loaded with sample data
- All services (Firebase, Grafana, dashboard) were started

Step 2: Warm-Up Phase

- System was run with a small number of users before testing
- This avoids inaccurate results due to startup delays

Step 3: Monitoring

- Grafana and other tools tracked system performance
- CPU, memory, and response times were observed

Step 5: Data Collection

- Results were recorded (response time, errors, throughput)

Step 6: Cool Down

- System was allowed to stabilize before next test

4.3 Workload Distribution

To make testing realistic, user activities were divided as follows:

- 60% Read operations → Viewing dashboard data
- 25% Write operations → Updating inventory
- 15% Advanced tasks → Analytics & digital twin

This distribution reflects real warehouse usage where most users view data, some update data, and a few use advanced features.

5. Results and Analysis

The obtained results from performance testing reveal the effectiveness of the proposed cloud-based warehouse management system when it comes to performing tasks efficiently during normal load and even moderate load, maintaining stability, speed, responsiveness, and synchronization of real-time data.

During the tests, the cloud-based system demonstrated an ability to work effectively with simultaneous access of multiple users to the dashboard, updating of inventory data, and interaction with the cloud.

When functioning under low to medium loads, the system exhibited fast responsiveness and flawless operation; thus, it can be concluded that the integration of the frontend dashboard with the Firebase database and monitoring has been optimized successfully. In addition, the Grafana monitoring tool operated flawlessly and did not increase overhead.

With the rise in the number of users at one time, the system continued operating within reasonable boundaries until a particular threshold was reached. As the system reached its threshold, where the highest level of load and stress was experienced, there was an increment in the response time. This signifies the performance drop in the system as it struggles to handle a large number of concurrent users. In this regard, server resources like the CPU and memory, as well as network traffic from the client and the backend to the cloud database, were stressed.

However, the error rate was reasonably low throughout the testing process. In addition, data consistency was maintained by the real-time database of Firebase while executing the reading and writing operations concurrently. Also, there were no significant malfunctions of the digital twin dashboard and analytics part of the system.

It is also essential to note the effect of network conditions on the performance of the system. Specifically, any increase in network latency or reduction in the quality of connection had a considerable effect on the performance, especially during real-time data synchronization and dashboard update processes. It is clear from the above discussion that this system relies heavily on stable network connection, something that is quite common for cloud-based applications.

Generally, the outcomes of the performance test prove the effectiveness of the system as being able to accommodate real-world warehouse environment operation using several users and real-time data processing. Although good performance is observed under normal circumstances, the necessity for further improvements to make the system scalable under heavy traffic remains evident.

6. Performance Optimization and Recommendations

The following section will present the analysis and optimization of the system after performance testing. The performance improvement techniques used in this project will be discussed along with suggestions for future improvements.

1. Database Optimization (Firebase)

- Queries were optimized to reduce unnecessary data fetching
- Indexing was applied to improve data retrieval speed

2. Dashboard Optimization

- Reduced unnecessary UI re-rendering
- Improved component efficiency

3. Connection Optimization

- Reduced number of active connections to Firebase
- Improved real-time synchronization efficiency

4. Monitoring Optimization (Grafana)

- Adjusted refresh rates of dashboards
- Reduced excessive data polling

5. Backend Performance Improvement

- Improved API handling and request processing
- Reduced delays in data processing

6.2 Recommendations for Future Improvements

Even though the system performs well, further improvements are recommended for large-scale deployment:

1. Load Balancing

- Distribute traffic across multiple servers

Benefit: Handles more users without slowing down

2. Caching

- Store frequently used data temporarily

Benefit: Reduces database load and improves speed

3. Cloud Scaling

- Use auto-scaling in cloud platforms

Benefit: Automatically handles high user traffic

4. API Optimization

- Reduce unnecessary API calls
 - Improve data transfer efficiency
- Benefit: Faster communication between components

5. Content Delivery Network (CDN)

- Deliver frontend assets faster using distributed servers
- Benefit: Faster loading for users in different locations

7. Challenges and Solutions

In the course of the testing procedure, several practical obstacles arose which influenced the quality of testing results, namely accuracy, stability, and reliability. The main obstacle in the course of the testing process was connected with the peculiarities of cloud services, especially Firebase. Due to the nature of cloud services that have limitations in terms of number of requests per second, there were cases of throttling. In other words, due to artificial limitations of cloud services, the system showed poor results, which were not associated with its capabilities but with the external limitations. To resolve this issue, it was decided to limit the number of requests per second and, if needed, use local/emulator environments to achieve better results.

Another major obstacle was the question of testing tool stability. While JMeter is a highly efficient tool that is usually used for testing systems' performance under load conditions, there were situations when particular components were unstable, especially WebSocket or real-time communication testing. Therefore, it was decided to apply other methods and check results in case of discrepancy.

There were differences in the performance of the application across different settings, particularly for the frontend aspects like the dashboard and digital twin visualization. The variations were caused by differences in browser performance and system configuration, which affected the speed and efficiency of the application. For consistency in the outcomes, testing was done in a controlled setting with a uniform browser.

A significant challenge involved the performance of the system during lengthy testing sessions. In testing for an extensive period, there was a steady rise in the utilization of memory. This could mean inefficient utilization of the resources by the system. This problem was addressed through an examination of the source code of the system, which was then improved.

Moreover, some data were also collected during testing using monitoring tools like Grafana, and at times, it also caused a strain on the system leading to an increase in its load. In order to counteract this issue, the monitoring configuration was optimized through the modification of refresh intervals and reduction in unnecessary data collection.

Overall, this served as a learning experience for both the system and the testing process. Through this exercise, several improvements were brought about and therefore enhanced the accuracy of the test results.

8. Ethical and Compliance Considerations

Ethics and compliance are vital in performance testing, particularly in the case of cloud solutions and applications based on the utilization of data. All actions associated with the testing of the warehouse management system integrated with cloud computing were performed carefully to prevent damage to the user, to the system, and other outside systems.

The most important ethical issue that was paid attention to was the security of personal data. Performance testing did not utilize any private or sensitive information; on the contrary, it used sample data for simulating the activities in the warehouse. Thus, personal information was not endangered throughout the entire testing procedure.

Another vital issue that had to be considered was the proper use of cloud resources. As the system utilizes cloud technologies such as Firebase, it was necessary to make sure that all testing procedures would not breach service agreements or lead to increased loading of common resources. Load testing was carried out within reasonable limits and did not exceed

normal traffic levels. Thus, ethical use of cloud solutions and adherence to their providers' policies were guaranteed.

Security issues were considered when conducting testing. The system uses role-based access control (RBAC) to guarantee that users have access only to certain features of the app. Performance testing was carried out without any attempts at bypassing the implemented access control mechanisms.

Concerning the legal requirement, the system design and test methodology comply with the principles of data protection, which include the data protection regulations mentioned, such as GDPR and PDPA of Sri Lanka. In spite of the absence of real personal data, the design of the system guarantees safe data transfer due to the encryption and access control capabilities.

Moreover, the principle of academic honesty was observed during the testing process and report writing. All used tools, methodologies, and technologies were cited correctly, and the present study is an original contribution made by the author personally.

Overall, ethical and compliance considerations were carefully followed to ensure that the performance testing process was conducted responsibly, securely, and in alignment with both technical standards and legal requirements.

9. References

- [1] S. Kirešová, M. Guzan, B. Fecko, O. Somka, V. Rusyn and R. Yatsiuk. [Online]. Available:
<https://ieeexplore.ieee.org/document/10402486?denied=>.
- [2] R. Ajax. [Online]. Available:
[https://www.researchgate.net/publication/388780068_How_Real Time Dashboards Improve Business Performance](https://www.researchgate.net/publication/388780068_How_Real_Time_Dashboards_Improve_Business_Performance).
- [3] S. Akhtar. [Online]. Available:
<https://ejournals.org/bjms/wp-content/uploads/sites/21/2025/05/Creating-User-Friendly-Dashboards.pdf>.
- [4] A. Sharma. [Online]. Available:
<https://www.sciencedirect.com/science/article/pii/S2452414X22000516>.
- [5] M. F. W. W. Metzger. [Online]. Available:
https://www.fh-wedel.de/fileadmin/Mitarbeiter/Records/Metzger_2023_-_A_Systematic_Literature_Review_on_Digital_Twins.pdf.
- [6] N. Gallagher. [Online]. Available:
<https://www.ibm.com/think/topics/digital-twin>.
- [7] A. P. Singh. [Online]. Available:
[https://www.researchgate.net/publication/367451523_A_Data_Visualization Tool- Grafana](https://www.researchgate.net/publication/367451523_A_Data_Visualization_Tool-Grafana).
- [8] T. Leppänen. [Online]. Available:
https://www.theseus.fi/bitstream/handle/10024/512860/Turkuamk_Bachelors_Thesis_Leppanen_Tiia.pdf?sequence=2&isAllowed=y.